

Implement Client Script and UI Policy

1. INTRODUCTION

This technical project document outlines the implementation of custom client-side dynamics on the Incident form in ServiceNow, focusing on UI Policies and Client Scripts to enforce data integrity and improve operational workflows.

Project Team

Evaangelin V Team Leader	Jagatheeswari V Team Member
Nirmala M Team Member	Swathi KM Team Member

2. PROJECT OVERVIEW

Purpose: The primary objective of this project is to implement Client Scripts and UI Policies within the ServiceNow platform to dynamically control field behaviors on the Incident form. By enforcing conditional mandatory attributes, read-only controls, and key validation checks, the project guarantees standardized data entry practices during record creation and modification.

Key Features

- UI Policy Implementation:** Configured the *High Impact Control* UI Policy on the Incident table.
- Automated Field Adjustments:** Automatically updates field urgency when High Impact conditions are met.
- Form Validation:** Prevents record saving if the Assigned To field is missing on High Impact incidents.
- List Edit Restriction:** Restricts direct state modification via list editing to preserve process workflows.
- Verification & Testing:** Executed functional validation using dedicated Incident test records.

3. ARCHITECTURE

Because this application is built on the enterprise ServiceNow platform, traditional MERN stack architecture (MongoDB, Express, React, Node) does not apply. Instead, ServiceNow’s native application architecture governs data flow and client-side execution.



Architectural Elements

- Client Scripts:** Execute on the client browser in response to user events (e.g., field change, form load, submit).
 - Auto set urgency for High Impact
 - Prevent save if assigned to missing
 - Prevent state change via list edit
- UI Policy:** Defines declarative rules (*High Impact Control*) for field behavior without hardcoded scripts.
- Database Entity:** Operates directly on the native Incident [incident] table, leveraging key fields: Impact, Urgency, Short Description, Assigned To, and State.

4. SETUP INSTRUCTIONS

Prerequisites: Active Internet connection, modern web browser, ServiceNow account, and access to a provisioned ServiceNow Developer Instance within the ServiceNow Lab environment.

Accessing the Instance

- **Access Channel:** Initiated via SkillWallet platform and shared direct links.
- **Developer Platform:** Navigated to the standard ServiceNow Developer Platform login page.
- **Authentication Email:** Authenticated using the registered *Naan Mudhalvan* student/developer email ID to access the assigned instance.

5. PROJECT STRUCTURE

Unlike file-system-based web applications with separate frontend and backend directories, this project is contained within the ServiceNow metadata structure.

Component Type	Identifier / Target	Details
Primary Table	Incident [incident]	Core ITSM table holding record data.
UI Policy	High Impact Control	Conditional visibility & mandatory attributes.
Client Scripts	3 Custom Scripts	Handles form submission, field changes, & list edit.
Tracked Fields	Impact, Urgency, State, etc.	Form fields subject to client behavior rules.

6. RUNNING THE APPLICATION

Execution takes place within the live ServiceNow Personal Developer Instance (PDI):

1. Log in to the requested ServiceNow instance using registered credentials.
2. Navigate to **System Definition** → **UI Policies** and configure *High Impact Control*.
3. Navigate to **System Definition** → **Client Scripts** and activate the three custom scripts.
4. Open the **Incident** module and click *Create New* or open existing records.
5. Modify the **Impact** field to test dynamic urgency changes and submission validations.

7. API DOCUMENTATION

Note on REST APIs: This project relies entirely on ServiceNow's built-in client-side engine and form scripts. No custom REST or backend API endpoints were created or exposed. Platform-native APIs (such as `g_form`) are utilized internally.

8. AUTHENTICATION

Authentication and session management are handled directly by ServiceNow platform security. Access control relies on instance authentication tied to the *Naan Mudhalvan* registered credentials. No independent user auth system was developed.

9. USER INTERFACE & SCRIPT BEHAVIOR

The standard ServiceNow Incident form serves as the UI layer. Implemented behaviors trigger during user interactions:

- **Auto set urgency for High Impact:** Dynamically sets the **Urgency** field when **Impact** is marked as High.

10. TESTING

System testing was conducted within the live instance using structured scenario validation across two generated test Incident records.

Script / Rule Name	Test Case Objective	Observed Result
Auto set urgency	Select High Impact on Incident form.	Urgency automatically shifts to High without manual input.
Prevent save validation	Attempt save on High Impact incident with empty Assigned To.	Form submission blocked; error banner displayed as expected.
Prevent list state edit	Double-click State field in Incident list view to edit.	Operation aborted with instructions to open full form.
High Impact UI Policy	Verify field visibility/mandatory toggles on condition match.	Fields dynamically adapt based on UI Policy condition rules.

11. SCREENSHOTS OR DEMO

- **Demonstration Video:** A 1-minute 25-second walkthrough video was recorded showcasing all live UI Policy triggers and Client Script validation alerts.
- **Repository:** All project artifacts, script code listings, and demo assets are uploaded and maintained in the project GitHub repository.

12. KNOWN ISSUES

No functional bugs or configuration errors were encountered during final testing in the ServiceNow Lab instance. The scripts operate within their intended scope on the Incident `[incident]` table.

13. FUTURE ENHANCEMENTS

- **Additional UI Policies:** Expand rules to cover dynamic visibility across Problem and Change forms.
- **Advanced Client Validations:** Add regex pattern matching on the Short Description or custom text fields.
- **Enhanced Incident Automation:** Automate priority recalculation using matrix lookups combining Impact, Urgency, and Category.
- **Role-Based Controls:** Restrict field editing capabilities based on user roles (e.g., ITIL Admin vs. End User).

PROJECT SUMMARY

This project successfully demonstrates the implementation of ServiceNow client-side customization. By leveraging one UI Policy and three targeted Client Scripts, the solution enforces data entry standardizations, prevents critical missing assignments on severe incidents, and protects lifecycle state integrity directly on the Incident table.